Lab 4 AVCV 2026/2027: Fire-risk classification from satellite images

CUSTOM and FINE-TUNING networks on the FireRisk subset

Daniel Truzman (100475161) César Álvarez-Cascos (100475191) • CodaBench team `100475161_100475191`

1. INTRODUCTION

The goal of this lab is to assign one of seven FireRisk classes (Water, Very Low, Low, Moderate, High, Very High and Non-burnable) to a single RGB satellite image. It is an image-classification problem, not fire detection: the label describes the fire risk of the area, not whether a fire is visible. The task has two main difficulties. The classes are strongly imbalanced, from 2.4% of the images for Water to 31% for Very Low, so a model that ignores the rare classes can still look accurate; this is why the lab uses macro-F1, which weights every class equally. Moreover, the five risk levels are ordered and look alike, and neighbouring levels such as Low, Moderate and High turn out to be the main source of errors.

We work with a subset of FireRisk [1] that contains 5,861 images of 320×320 pixels. It is split in a stratified way into 3,516 training, 1,172 validation and 1,173 test images with hidden labels (class counts in Table 3). The networks are trained on 224×224 crops of these images (Section 2.1).

The CUSTOM system is a VGG-style CNN trained from scratch and refined through a staged random search; the FINE-TUNING (FT) system adapts ImageNet-pretrained torchvision models, of which Swin-T was selected. The experiments tried to answer a few concrete questions: whether the network from scratch was limited by capacity or by the data, how class weighting and augmentation affect the minority classes, and whether ensembles or retraining on train+val would help.

2. METHODOLOGY

2.1. Preprocessing and data augmentation

Images are read as RGB tensors in $[0, 1]$ and normalised with the ImageNet statistics, which the pretrained backbones require; CUSTOM uses the same pipeline. During training, each 320×320 image is randomly cropped to 224×224, and every batch is then augmented on the GPU with horizontal and vertical flips, 90° rotations and random changes of gamma, brightness, contrast and saturation. The augmentation has a *light* and a *strong* preset; the search chose *strong* for CUSTOM and *light* for Swin-T. Validation images are centre-cropped to 224×224, the setting used for model selection. For the test predictions, however, we feed the whole 320×320 image, which both networks accept because they end in global pooling: although they were trained and validated at 224×224, in our experiments the full image gave better results on the test set. No test-time augmentation (TTA) is applied.

An aerial image has no natural "up", so flips and 90° rotations create new samples without changing the label. Colour is different, since the vegetation is probably the main visual cue for fire risk, so the colour changes were kept mild. We also tried mixup [2], CutMix [3] and TTA, but none of them improved validation macro-F1 by more than the seed noise (Section 2.4).

2.2. System design: CUSTOM network

The CUSTOM network (Table 5) is built only from primitive layers: four VGG-style blocks [4] of 48 to 384 channels, each with two Conv 3×3–BatchNorm [5]–ReLU layers and a 2×2 max-pooling, followed by global average pooling (GAP) [6] and a small classifier with dropout, for 2.74 M parameters.

We reached this design step by step from the reference notebook, keeping a change only when it improved validation macro-F1. The first one was to replace the Flatten head, which held almost all the parameters of the network, by GAP. We then used two 3×3 convolutions with BatchNorm in each block instead of a single 5×5 one, which covers the same area with fewer weights and trained more stably. Finally, since the network was still underfitting, we made it wider (Section 2.4).

2.3. System design: FINE-TUNING network

We kept the provided AlexNet [7] as a baseline and compared it with three more recent backbones with torchvision ImageNet-1k weights [8]: ConvNeXt-T [9], Swin-T [10] and EfficientNetV2-S [11]. EfficientNetV2-S was unstable with mixed precision (its loss became non-finite), so we replaced it by ResNet-50 [12]. In every model the original classifier is replaced by a new linear layer with 7 outputs, and the adaptive pooling by a simple mean so that training stays deterministic.

We did not freeze any layer: satellite images are quite different from ImageNet photographs, so the whole network has to adapt. To avoid destroying the pretrained features, the backbone uses a small learning rate and the new head the same rate or ten times more [13], and the inputs follow the pretraining conditions (224×224, ImageNet

normalisation). Swin-T obtained the best validation macro-F1 (0.546) and overfitted less than ResNet-50, so it is our FT model.

2.4. Training protocol and model selection

Table 1: Training configuration of the selected models.

	CUSTOM (n07_60ep)	FT (Swin-T, swin_p2_1)
Loss	Cross-entropy, class weights $\propto 1/\sqrt{n_c}$, no label smoothing, no mixup	
Optimiser & schedule	AdamW [14]; linear warm-up + cosine decay [15]; fp16 mixed precision	
Learning rate	$3.17 \cdot 10^{-4}$	backbone $2.37 \cdot 10^{-5}$, head $\times 1$
Weight decay	0.0464	0.0395
Batch / warm-up	128 / 1 epoch	32 / 2 epochs
Epochs (selected epoch)	60 (36)	30 (26)
Augmentation	strong	light, drop path 0.2
Common	Seed 42; full train split (no subsampling); Kaggle Tesla T4; PyTorch 2.11 [16]	

Table 1 gives the final configuration of both models. To reach it we searched in phases, going from broad to specific.

Why random search. We had about ten hyperparameters to tune (architecture, learning rate, weight decay, batch size, augmentation, class weights, ...). A grid search would need too many runs even with two or three values per hyperparameter, and most of them would be spent varying hyperparameters that barely matter. For the same budget, a random search tries many more different values of each hyperparameter [17]. We also stopped early the trials that were clearly below the median of the previous ones [18], to spend the time on more promising configurations.

CUSTOM. In phases 1 and 2 (about 40 short runs of 15 epochs, split between two computers) we first chose the architecture and the optimiser, and then secondary settings such as the class weights and the regularisation. In phase 3 the three best configurations were trained for 40 epochs, and all of them got stuck at about 0.45–0.46 with almost equal training and validation scores. This told us that the network was underfitting, so instead of tuning further we changed approach: in phases 4 and 5 we tested one idea at a time. Widening the network and training for 60 epochs were the only clear improvements, whereas inverse-frequency class weights made the results worse and more capacity (more channels or blocks) did not help. Since repeating a run with another seed already changes the score by about 0.01, we only accepted changes clearly above that. A final, broader search of 121 trials found a model with a higher validation score, but it did not hold on the public test, so we kept the simpler network.

FINE-TUNING. For each backbone we tried 10 configurations for 10 epochs and retrained the best two for 30 epochs.

In every run we keep the epoch with the best validation macro-F1, without early stopping. Training is deterministic (fixed seeds and deterministic algorithms), so the notebook reproduces the selected models on the same GPU (Tesla T4).

3. RESULTS AND ERROR ANALYSIS

Our main metric is macro-F1, the mean of the seven per-class F1 scores. All model choices were made on validation macro-F1, with training F1 only as a diagnostic; the public leaderboard was only used to discard the deep-search model and to choose the 320×320 test input.

Experiments. Table 2 summarises the main experiments. For CUSTOM, the improvement from the reference ($\approx$0.36) to our final network (0.502) came from the GAP head, the extra width and the longer training. Other ideas, such as CutMix, ensembles, per-class decision thresholds or retraining on train+val, gave no clear gain. The thresholds are a good example: they raised the score on the validation set, but almost nothing under cross-validation, which warned us about overfitting the validation set. For FT, all modern backbones beat AlexNet (0.452), and Swin-T reached 0.546.

Learning curves. For CUSTOM, the training and validation curves in Figure 3 stay close, and validation F1 saturates at 0.49–0.50 after epoch 30: the model does not overfit, yet it stops improving. Swin-T fits the training set much better (training F1 0.77 at the selected epoch) but improves validation by only +0.04, so most of that extra fit does not transfer.

Per-class results and confusions. In Table 3, Water (F1 0.85) and Non-burnable (0.80) are easy to recognise, while the intermediate levels are hard: Low (0.22), Moderate (0.32) and High (0.33) for CUSTOM, and the same three for FT (0.33, 0.38 and 0.41). Precision and recall are similar for all classes except Low, of which CUSTOM recovers only 19%. The confusion matrices (Figure 1) show where they go. Most errors happen among the five risk levels (77% of CUSTOM errors and 76% of FT errors), and about half of all errors fall between two adjacent levels (49% and 52%). Low is the clearest case: CUSTOM labels 40% of the Low images as Very Low, and even

Swin-T sends 35% of them there. Very Low is in turn often mistaken for Non-burnable (13% and 11%).

The examples in Figure 2 suggest that part of these errors cannot be solved from the image alone. The Low image is mostly open water and both models predict Water, while the Very High image shows pale, almost bare ground that both models read as Very Low. Since the labels come from a hazard map much coarser than the image patch [1], what makes an area risky is often not visible in it. Consistently, CUSTOM reaches only 0.53 (Low 0.30) on its own training images without augmentation.

Competition feedback. On the public leaderboard (Table 4), our final submission (ID 1042782) scores 0.50 for CUSTOM and 0.49 for FT. The final CUSTOM network keeps its validation score (0.502), whereas the models chosen after the longest searches lose the most: Swin-T drops from 0.546 to 0.49, and the best model of the 121-trial search from 0.524 to 0.47. This points to selection bias from reusing the same validation images across hundreds of trials, plus the noise of a small public subset.

4. CONCLUSIONS

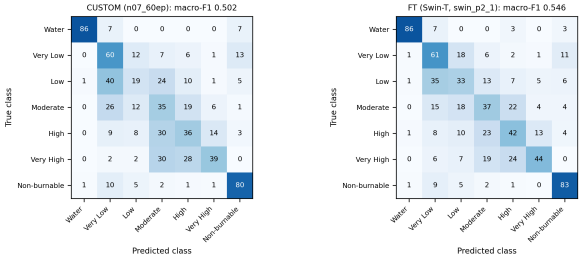
The changes with the largest supported effect were different for each system. For CUSTOM, they were those that addressed underfitting: the GAP head, the extra width and the longer schedule raised validation macro-F1 from 0.44 to 0.50, while regularisation such as CutMix did not help. For FT, the decisive factor was the pretraining itself: with a small backbone learning rate, Swin-T reached 0.546 in 30 epochs. Fine-tuning is therefore clearly better on validation, but on the public subset the two systems end up almost level (0.50 vs 0.49), so with this amount of data a well-designed network trained from scratch, with ten times fewer parameters, remains a competitive and much cheaper option.

The main limitation lies in the data rather than in the models. The intermediate classes remain hard even on training images, in line with the 61–65% accuracy reported with about 91k images in the original paper [1], and reusing one validation set inflated the scores of the most searched models by up to 0.06. The next steps follow from these problems: k-fold cross-validation for a more reliable selection than a single reused split; a loss that uses the order of the classes, since most errors fall between adjacent levels; and remote-sensing pretraining and a higher input resolution, to extract more visual information.

Individual contributions.

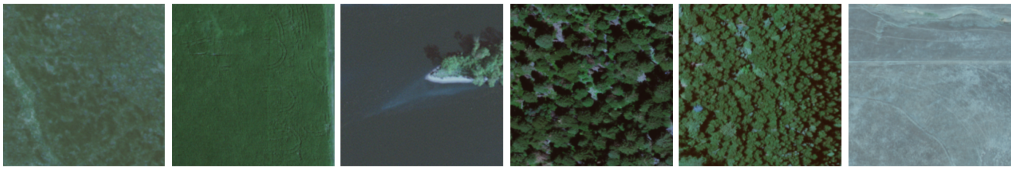
- **César Álvarez-Cascos (100475191):** design of CUSTOM search phases 1–3 (two-machine random search, finalists and the 121-trial deep search) and the GPU speed-up of training (images preloaded in RAM, batch augmentation on the GPU, mixed precision).
- **Daniel Truzman (100475161):** design of CUSTOM phases 4–5 (widening to 48×2, 60 epochs, selection of n07_60ep, deterministic Kaggle runs) and the CutMix, ensemble, threshold and train+val experiments.
- **Both:** the training runs on both members' machines, the notebook and the fine-tuning search.

Use of AI tools. We used an AI assistant as a productivity tool, always following our own criteria and instructions once we had defined the design criteria ourselves. It helped us implement and debug code, look for errors, suggest possible improvements and edit the text of this report. Every experiment, result and decision was run, checked and taken by the authors.



(a) CUSTOM **(b)** FT (Swin-T)

Figure 1: Validation confusion matrices (224 × 224 centre crop), row-normalised (%); rows: true class, columns: predicted class.



True	Water	Very Low	Low	Moderate	High	Very High
CUSTOM	Very Low	Non-burnable	Water	Very High	Very High	Very Low
FT	High	Non-burnable	Water	Very High	Moderate	Very Low

Figure 2: Validation errors shared by both models: per true class, the one with the most confident FT prediction (chosen automatically).

Table 2: Main experiments: validation macro-F1 at the best epoch (L10: mean of the last 10 epochs). Reference and selected models in bold.

Cat.	Change	Objective	Budget	Val F1
CUSTOM	**Reference notebook (StepLR)**	Baseline	60 ep	≈0.36
CUSTOM	GAP instead of Flatten (first version)	Fewer FC parameters; train F1 ≈0.42	30 ep	0.440
CUSTOM	Phases 1–3 random search	Architecture, optimiser, regularisation	15 ep (finalists 40)	0.45–0.465
CUSTOM	Inverse-frequency class weights	Help minority classes	40 ep	L10 0.431 vs 0.453
CUSTOM	48 ch, 2 convs/block (p4_09_B)	More capacity (underfitting)	40 ep	0.496 (L10 0.483)
CUSTOM	64 ch; 5 blocks; crop 288/eval 320	More capacity / resolution	40 ep	no gain
CUSTOM	**60 epochs (n07_60ep)**	Longer schedule	60 ep	**0.502** (L10 0.489)
CUSTOM	CutMix (standard; risk-only)	Regularisation	80 ep	L10 0.483; 0.470 vs 0.486
CUSTOM	Deep search (F3_B_038)	Joint search, 121 trials	80 ep, input 320	0.524
FT	**AlexNet (provided recipe)**	Baseline	5 ep	0.452
FT	ConvNeXt-T (cnx_p2_0)	Modern CNN backbone	30 ep	0.505
FT	ResNet-50 (r50_p2_0)	Residual backbone (train F1 0.99)	30 ep	0.539
FT	**Swin-T (swin_p2_1)**	Transformer backbone	30 ep	**0.546**

Table 3: Class counts and per-class validation precision / recall / F1 of the selected models (224 × 224 centre crop, as used for model selection).

Class	Train	Val	CUSTOM P/R/F1	FT P/R/F1
0 Water	86	29	.83/.86/.85	.81/.86/.83
1 Very Low	1,088	363	.59/.60/.60	.65/.61/.63
2 Low	535	178	.27/.19/.22	.33/.33/.33
3 Moderate	431	144	.29/.35/.32	.39/.37/.38
4 High	315	105	.31/.36/.33	.39/.42/.41
5 Very High	163	54	.41/.39/.40	.41/.44/.43
6 Non-burnable	898	299	.79/.80/.80	.80/.83/.81
Total / macro	3,516	1,172	.500/.508/**.502**	.539/.553/**.546**
Accuracy			.535	.577

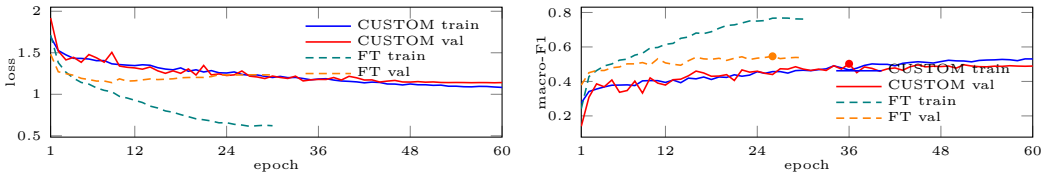
Table 4: Public CodaBench macro-F1 (fixed undisclosed test subset). Bold: final submission with the selected models; n/r: not recorded.

ID	Date	Models (custom / ft)	CUSTOM	FT
1036062	05 Nov	p4_09_B / placeholder	0.49	–
1038020	06 Nov	n07, train+val / –	0.49	–
1038186	06 Nov	F3_B_038 / F1_B_037	0.47	0.47
1038323	06 Nov	CUSTOM ens. (5) / + TTA	0.48	n/r
1042782	**08 Nov**	**n07_60ep / swin_p2_1**	**0.50**	**0.49**

Table 5: Final CUSTOM architecture.

Stage	Layers	Output
Input	RGB crop	3 × 224 × 224
Blocks 1–4	[Conv3×3-BN-ReLU]×2, MaxPool 2×2	48/96/192/384 ch
	(spatial size 112/56/28/14)	384 × 14 × 14
GAP	spatial mean	384
Head	FC 384→256, ReLU, Dropout 0.3, FC 256→7	7

2,739,447 parameters; Kaiming initialisation.



(a) Loss **(b)** Macro-F1

Figure 3: Train (with augmentation) and validation curves of the selected models; dots mark the selected epochs (CUSTOM 36, FT 26).

REFERENCES

[1] Shuchang Shen, Sachith Seneviratne, Xinye Wanyan, et al. FireRisk: A remote sensing dataset for fire risk assessment with benchmarks using supervised and self-supervised learning. In *DICTA*, 2023.

[2] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, et al. mixup: Beyond empirical risk minimization. In *ICLR*, 2018.

[3] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, et al. CutMix: Regularization strategy to train strong classifiers with localizable features. In *ICCV*, 2019.

[4] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[6] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *ICLR*, 2014.

[7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012.

[8] Olga Russakovsky, Jia Deng, Hao Su, et al. ImageNet large scale visual recognition challenge. *IJCV*, 2015.

[9] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, et al. A ConvNet for the 2020s. In *CVPR*, 2022.

[10] Ze Liu, Yutong Lin, Yue Cao, et al. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021.

[11] Mingxing Tan and Quoc V. Le. EfficientNetV2: Smaller models and faster training. In *ICML*, 2021.

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, et al. Deep residual learning for image recognition. In *CVPR*, 2016.

[13] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *ACL*, 2018.

[14] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[15] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.

[16] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[17] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *JMLR*, 2012.

[18] Daniel Golovin, Benjamin Solnik, Subhodeep Moitra, et al. Google vizier: A service for black-box optimization. In *KDD*, 2017.